Cache Controller with Snoopy Coherence for a Dual-Core System

Structure of Computer Systems · Project documentation

Student: Arthur Kantor

Group: 30232

Coordinating teacher: Mădălin Neagu

Academic year: 2026–2027

Technical University of Cluj-Napoca

Faculty of Automation and Computer Science, Computer Science Department

Table of Contents

- **1. Introduction**
  - 1.1 Project Proposal
  - 1.2 Project Plan
- **2. Bibliographic Study**
  - 2.1 Cache Memory Organization
  - 2.2 Replacement Policies
  - 2.3 Write Policies
  - 2.4 The Cache Coherence Problem
  - 2.5 Snoopy Coherence and the MSI Protocol
  - 2.6 Shared Bus and Arbitration
  - 2.7 Modeling and Simulation in VHDL
- **3. Bibliography**

1. Introduction

1.1 Project Proposal

Project Description

The project designs, describes in VHDL and verifies by simulation a cache controller for a dual-core system. Each core has a private write-back cache; the two caches share one bus to main memory and are kept coherent by the MSI snoopy protocol.

Private caches keep frequently used data close to each core, but they also create several copies of the same memory line. Without a coherence mechanism, one core could keep reading a stale value after the other core has written it. The project models, at register-transfer level, how a cache controller serves its own core while watching the bus to keep both copies consistent.

Each core sends its reads and writes only to its own cache controller; misses, upgrades and write-backs go out on the shared bus, where the other controller snoops them.

Specifications

Cache (one per core)

- All memory accesses of a core go through its cache.
- N-way set-associative organization; the number of sets, the associativity N and the line size are VHDL generics.
- Replacement policy LRU or FIFO, selected through a generic.
- Write-back policy, with write-allocate on write misses.

Coherence and interconnect

- MSI protocol: every cache line is in one of the states Modified, Shared or Invalid.
- Each controller snoops the shared bus and responds to read (BusRd), read-exclusive (BusRdX) and invalidate (BusUpgr) transactions, writing back a modified line when required.
- One shared bus to main memory, with an arbiter that grants it to one controller at a time (round-robin); a granted transaction completes before the next grant.

Verification

- The two cores are testbench request generators issuing read and write sequences.
- Simulation shows hits, misses, replacements and write-backs.

- Simulation shows three coherence scenarios:
  1. Both cores read the same line: both copies end in state S.
  2. One core writes a line cached by the other: the other copy is invalidated.
  3. One core reads a line modified in the other cache: the owner writes the line back and the requester receives it (write-back with cache-to-cache transfer).

Objectives

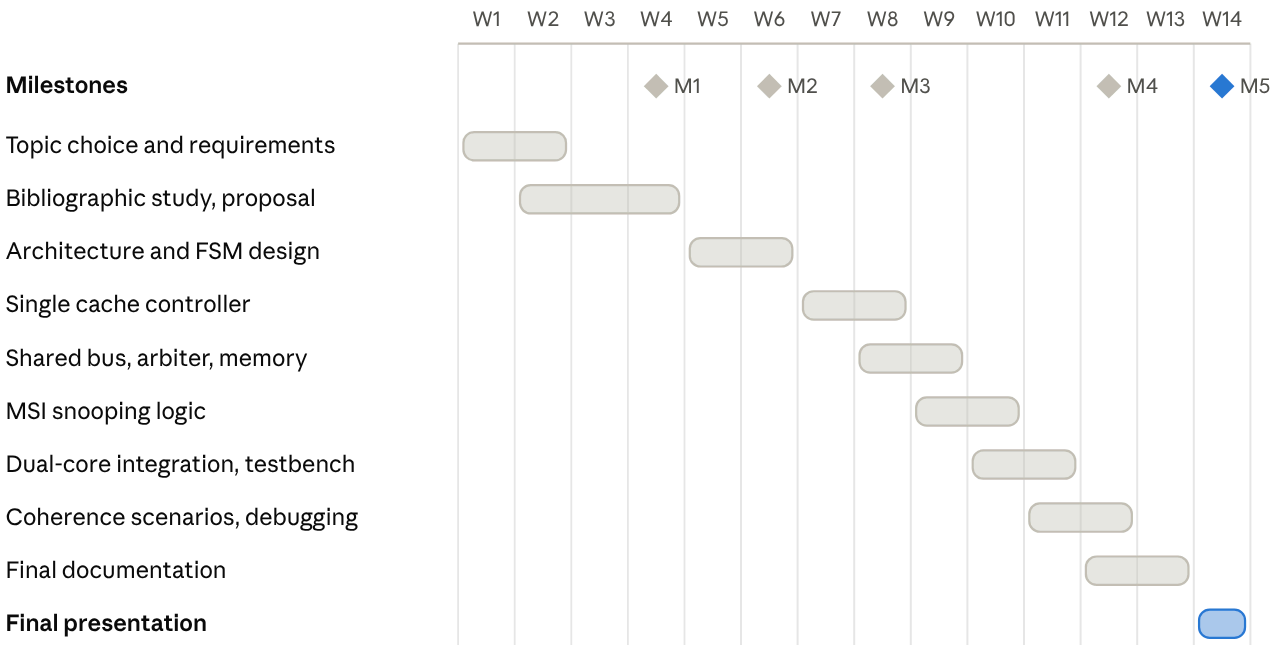
- **Correctness:** every read returns the value of the most recent write to that address, by either core.
- **Configurability:** changing the number of sets, N, the line size or the replacement policy only changes generic values.
- **Modularity:** cache storage, replacement logic, processor-side controller, snoop logic and arbiter are separate entities, so the design can later be extended (for example to MESI or more cores).
- **Observability:** each scenario has a self-checking testbench (assertions) and a waveform showing line states, hit/miss signals and bus transactions.

1.2 Project Plan

The work is split into five milestones over the 14-week semester: the coherent dual-core system is fully simulated by week 12, leaving the last two weeks for the final documentation and presentation.

Coherent dual-core system simulated by week 12, presented in week 14

Plan by semester week; diamonds mark milestones M1 to M5



Project timeline · 14 weeks, 5 milestones

Consecutive tasks overlap by one week, so each component is still being tested while the next one is started.

Milestone	Week	Deliverable
M1	W4	Documentation part 1: introduction, project proposal, bibliographic study, bibliography, project plan
M2	W6	Design: block diagram, entity interfaces, processor-side and snoop-side state machines, bus protocol
M3	W8	Single cache controller simulated: hits, misses, LRU/FIFO replacements, write-backs
M4	W12	Dual-core system simulated: shared reads, invalidation, write-back with cache-to-cache transfer
M5	W14	Final documentation and project presentation

2. Bibliographic Study

This chapter summarizes the concepts the design relies on: how a set-associative cache is organized, how it replaces and writes lines, and how a snoopy MSI protocol keeps two private caches coherent over a shared bus.

2.1 Cache Memory Organization

A cache keeps copies of main-memory blocks, called lines, close to the processor, exploiting temporal and spatial locality [1], [5]. In an N-way set-associative cache the storage is divided into S sets of N lines; a memory line may be placed in any of the N ways of exactly one set. Direct-mapped (N = 1) and fully associative (S = 1) caches are the two extremes [2].

For an A-bit address, a line size of L bytes and S sets, the cache capacity is C = S × N × L bytes, and the address is split into three fields:

- **Offset:** log2(L) bits, selecting the byte inside the line.
- **Index:** log2(S) bits, selecting the set.
- **Tag:** A − index − offset bits, identifying the line inside the set.

For example, a 32-bit address with S = 64, N = 4 and L = 16 B gives a 4 KiB cache with 4 offset bits, 6 index bits and 22 tag bits. On an access, the index selects a set and the N stored tags are compared in parallel; a match on a valid line is a hit. Each line holds its tag, its data, its coherence state and the information used by the replacement policy.

2.2 Replacement Policies

On a miss in a full set, one way must be evicted; an invalid way, when present, is always filled first [1].

- **LRU (least recently used)** evicts the line unused for the longest time. Exact LRU keeps a log2 N-bit age per way: the accessed way gets age 0 and every way younger than it ages by one. Its cost grows with N, so large caches use pseudo-LRU approximations [1].
- **FIFO (first in, first out)** evicts the line loaded earliest, using one log2 N-bit pointer per set, advanced only when a line is filled. It is cheaper, but it ignores reuse: a frequently accessed line can still be evicted [5].

Implementing both behind one generic allows their hit rates to be compared on the same access sequence.

2.3 Write Policies

On a write hit, a **write-through** cache updates memory on every write, while a **write-back** cache updates only the cache and marks the line dirty; memory is updated when the line is evicted [1]. Write-back sends far less traffic to memory, which matters when two cores share one bus [6].

On a write miss, **write-allocate** first brings the line into the cache and then writes it, while **no-write-allocate** writes directly to memory. Write-back is normally paired with write-allocate, so later writes to the same line hit [2]. Under MSI, the Modified state plays the role of the dirty bit.

2.4 The Cache Coherence Problem

With private write-back caches, two copies of one line can diverge: if both cores cache address X and core 0 writes it, core 1 keeps reading the old value from its own cache [3]. A coherence protocol prevents this by enforcing two invariants [3]:

- **Single writer, multiple readers:** at any moment a line is either writable in exactly one cache or read-only in any number of caches.
- **Data value:** a read returns the value of the most recent write to that line.

Protocols are either snooping or directory-based. Snooping broadcasts every request on a shared medium observed by all controllers; it is simple and fast for a few cores but does not scale. Directory protocols record the sharers of each line and scale to many cores [1], [3]. For two cores on one bus, snooping is the natural choice.

2.5 Snoopy Coherence and the MSI Protocol

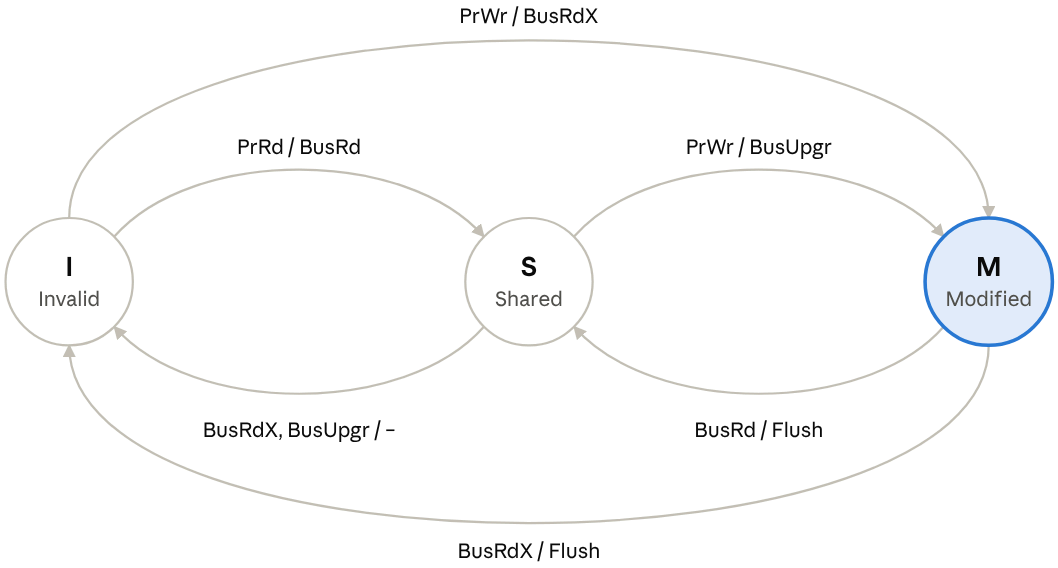
Goodman's write-once protocol first kept caches coherent by snooping the bus [6]. Later protocols are named after their states: MSI, MESI (the Illinois protocol [7]) and MOESI; Archibald and Baer compared them by multiprocessor simulation [8]. MSI is the basic invalidation-based protocol and the one used here, in the notation of Culler et al. [4].

State	Meaning	Other cache may hold	Memory up to date
M (Modified)	Only valid copy, written locally	Nothing (I)	No
S (Shared)	Clean, read-only copy	S or I	Yes

State	Meaning	Other cache may hold	Memory up to date
I (Invalid)	Line absent or invalidated	M, S or I	—

Bus transaction	Issued on	Response of the other cache
BusRd	Read miss	An M copy is flushed and demoted to S
BusRdX	Write miss (write-allocate)	Any copy is invalidated; an M copy is flushed first
BusUpgr	Write hit on an S line	The S copy is invalidated; no data is transferred
Flush	Owner supplies or evicts an M line	Memory is updated

A core writes only in state M; snooped requests demote the other copy



MSI state diagram · above: core requests · below: snooped bus transactions

Arcs above the states are requests from the local core, arcs below are transactions snooped on the bus; each label reads event / bus action. Hits keep their state (S on PrRd, M on PrRd and PrWr), and a snooped BusRd leaves an S copy unchanged. Evicting an S line drops it silently, while evicting an M line first writes it back (Flush).

When the owner flushes a line in response to BusRd or BusRdX, both memory and the requesting cache can take the data from the bus (cache-to-cache transfer), saving a second memory access [4]. A known cost of MSI is that reading and then writing an unshared line needs two transactions (BusRd, then BusUpgr); the Exclusive state of MESI removes the second one [7] and is outside the scope of this project.

2.6 Shared Bus and Arbitration

The bus is a single shared resource: each controller raises a request line and the arbiter asserts exactly one grant [2]. A fixed-priority arbiter is the simplest but can starve the low-priority core; a round-robin arbiter alternates between requesters and is fair.

The bus also serializes coherence: the order in which requests are granted is the order in which all caches see writes to a line [4]. With an atomic bus, where a transaction completes before the next grant, the main remaining race is two S copies requesting BusUpgr at once; the controller that loses arbitration observes the winner's invalidation and must reissue its request as BusRdX [4].

2.7 Modeling and Simulation in VHDL

The processor-side and snoop-side controllers map naturally onto finite-state machines, and the tag, data and state storage onto arrays of records indexed by set and way [9]. Generics parameterize the number of sets, the associativity and the line size, and generate statements build the parallel tag comparators [9], [10]. Testbench processes model the cores, and assertions check expected values and line states, so each coherence scenario verifies itself in simulation [9].

3. Bibliography

1. J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
2. D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Waltham, MA, USA: Morgan Kaufmann, 2014.
3. V. Nagarajan, D. J. Sorin, M. D. Hill and D. A. Wood, *A Primer on Memory Consistency and Cache Coherence*, 2nd ed., Synthesis Lectures on Computer Architecture. Morgan & Claypool, 2020.
4. D. E. Culler, J. P. Singh and A. Gupta, *Parallel Computer Architecture: A Hardware/Software Approach*. San Francisco, CA, USA: Morgan Kaufmann, 1999.

5. A. J. Smith, "Cache memories," *ACM Computing Surveys*, vol. 14, no. 3, pp. 473–530, 1982.
6. J. R. Goodman, "Using cache memory to reduce processor-memory traffic," in *Proc. 10th Annual International Symposium on Computer Architecture (ISCA)*, 1983, pp. 124–131.
7. M. S. Papamarcos and J. H. Patel, "A low-overhead coherence solution for multiprocessors with private cache memories," in *Proc. 11th Annual International Symposium on Computer Architecture (ISCA)*, 1984, pp. 348–354.
8. J. Archibald and J.-L. Baer, "Cache coherence protocols: Evaluation using a multiprocessor simulation model," *ACM Transactions on Computer Systems*, vol. 4, no. 4, pp. 273–298, 1986.
9. P. J. Ashenden, *The Designer's Guide to VHDL*, 3rd ed. Burlington, MA, USA: Morgan Kaufmann, 2008.
10. *IEEE Standard for VHDL Language Reference Manual*, IEEE Std 1076-2019, 2019.